

一、整合概述

iptables 是 Linux 内核级别包过滤规则配置的用户空间命令行工具。本文档记录了 iptables v1.8.7 (PHS 版本) 在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- iptables 版本：v1.8.7 (PHS 定制版，位于 `third_party/iptables`)
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux
- 目标架构：ARM64 (aarch64)
- 编译工具链：aarch64-v01c01-linux-musl-gcc (GCC)
- 编译日期：2026-05-18

关键背景：

- `third_party/iptables` 已包含完整移植版源码 (含 `extensions/libiptc/libxtables` 子目录、`genInit.py` 等)
- 该版本自带 `ohos_lite` 分支，支持小型系统 (small system) 编译

二、整合步骤

1. 确认源码目录

确认 `third_party/iptables` 包含完整源码：

```
ls third_party/iptables/extensions/BUILD.gn      # 必须存在
ls third_party/iptables/libiptc/BUILD.gn          # 必须存在
ls third_party/iptables/libxtables/BUILD.gn        # 必须存在
ls third_party/iptables/extensions/genInit.py     # 必须存在
```

若上述文件不存在，说明目录为 OHOS 空壳版本，需获取完整移植版源码替换。

2. 产品配置修改

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/config.json`

在 `thirdparty` 子系统的 `components` 数组中添加 iptables 组件：

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "zlib", "features": [] },
    { "component": "openssl", "features": [] },
    { "component": "libmnl", "features": [] },
    { "component": "busybox", "features": [] },
    { "component": "ethtool", "features": [] },
    { "component": "iptables", "features": [] }
  ]
}
```

`bundle.json` 中 `component.name` 已为 `"iptables"`，无需修改。

3. 修复 ARM64/musl 头文件兼容性 (`extensions/BUILD.gn`)

与 BusyBox 整合同理，iptables 的 `extensions/BUILD.gn` 中 `ext_config` 包含的 `-D_UAPI_LINUX_SOCKET_H` 和 `-D__kernel_sa_family_t=sa_family_t` 会导致 musl libc 的 `sockaddr_storage` 类型未定义。

文件: `third_party/iptables/extensions/BUILD.gn`

修改前 (第 34-36 行) :

```
"-D_GNU_SOURCE",
"-D_UAPI_LINUX_SOCKET_H",           # Prevent sockaddr_storage redefinition
"-D__kernel_sa_family_t=sa_family_t" # Define the type
```

修改后:

```
"-D_GNU_SOURCE",
```

修改原因: `-D_UAPI_LINUX_SOCKET_H` 宏会跳过 `linux/socket.h` 的包含，导致 `sockaddr_storage` 结构体未定义。在 ARM64 + musl libc 环境下，此宏必须移除。

4. 添加 sysroot 头文件搜索路径 (`extensions/BUILD.gn`)

iptables 自带的 `include/linux/netfilter/` 目录下存在部分头文件 wrapper，会遮蔽 sysroot 中包含完整结构体定义的头文件。通过将 sysroot include 路径置于 iptables include 路径之前解决。

文件: `third_party/iptables/extensions/BUILD.gn`

在所有 `include_dirs` 列表的最前面添加 sysroot 路径:

修改前:

```
include_dirs = [
  "//third_party/iptables",
  "//third_party/iptables/extensions",
  "//third_party/iptables/include",
  "//third_party/iptables/iptables",
  "//third_party/iptables/libiptc",
]
```

修改后:

```
include_dirs = [
  "$root_out_dir/sysroot/usr/include",
  "//third_party/iptables",
  "//third_party/iptables/extensions",
  "//third_party/iptables/include",
  "//third_party/iptables/iptables",
  "//third_party/iptables/libiptc",
]
```

此修改涉及 `ohos_lite` 分支下的 3 处 `include_dirs` (`libext`、`libext4`、`libext6`) 以及 `else` 分支下的 3 处 `include_dirs`，共 6 处。

5. 删除冲突的 wrapper 头文件

iptables 自带的部分 `include/linux/netfilter/*.h` 文件为空壳 wrapper，会遮蔽 sysroot 中包含真实结构体定义的头文件，导致编译错误。需删除以下 3 个文件：

```
cd third_party/iptables/include/linux/netfilter

# 删除空壳 wrapper
rm -v xt_connmark.h    # 空壳，遮蔽 sysroot 的 struct xt_connmark_tginfo2 定义
rm -v xt_mark.h        # 空壳，遮蔽 sysroot 的 struct xt_mark_tginfo2 定义
rm -v xt_dscp.h        # 自引用 wrapper，遮蔽 sysroot 的 XT_DSCP_MAX 定义
```

删除原因分析：

- 这些文件是简单的 `#include` wrapper，在 case-sensitive 的 Linux 文件系统上自引用导致 sysroot 版本被跳过
- sysroot 中同时存在大写和小写版本（如 `xt_CONNMARK.h` 和 `xt_connmark.h`），功能更完整
- sysroot 的 `xt_connmark.h`（678 字节）包含 `struct xt_connmark_tginfo2` 等完整定义，而 iptables wrapper（136 字节）只是空壳

6. 修复 connmark 枚举重复定义

源文件 `extensions/libxt_connmark.c` 本地定义了 `D_SHIFT_LEFT/D_SHIFT_RIGHT` 枚举，与 sysroot 的 `xt_connmark.h` 中的定义冲突。

文件：`third_party/iptables/extensions/libxt_connmark.c`

修改前（约第 34-38 行）：

```
enum {
    D_SHIFT_LEFT = 0,
    D_SHIFT_RIGHT,
};

enum {
    O_SET_MARK = 0,
```

修改后：

```
enum {
    O_SET_MARK = 0,
```

修改原因：`D_SHIFT_LEFT/D_SHIFT_RIGHT` 已由 sysroot 的 `<linux/netfilter/xt_connmark.h>` 提供，本地定义造成重复声明错误。直接删除本地的枚举定义即可。

7. 脚本权限修复

由于代码通过 Windows 文件系统挂载到 Docker 容器，脚本文件默认没有执行权限：

```
cd <源码根目录>/third_party/iptables
chmod +x extensions/genInit.py
chmod +x install.sh
```

三、问题解决记录

问题 1：脚本执行权限被拒绝

错误信息：

```
PermissionError: [Errno 13] Permission denied
```

原因：Windows/Linux 跨文件系统挂载，脚本文件默认没有 `+x` 权限。

解决方案：手动添加执行权限。

问题 2：sockaddr_storage 类型未定义

错误信息：

```
sysroot/usr/include/netinet/in.h: error: field has incomplete type 'struct sockaddr_storage'
```

原因：`-D_UAPI_LINUX_SOCKET_H` 宏跳过了 `linux/socket.h` 头文件，导致 `sockaddr_storage` 结构体定义缺失。ARM64 + musl libc 环境对此更敏感。

解决方案：从 `extensions/BUILD.gn` 的 `ext_config` 中移除 `-D_UAPI_LINUX_SOCKET_H` 和 `-D_kernel_sa_family_t=sa_family_t`。

问题 3：struct xt_connmark_tginfo2 不完整类型 / XT_DSCP_MAX 未声明

错误信息：

```
error: invalid application of 'sizeof' to incomplete type 'struct xt_connmark_tginfo2'
error: 'XT_DSCP_MAX' undeclared here (not in a function)
```

影响文件：`libxt_connmark.c`、`libxt_dscp.c`、`libxt_mark.c`

根因分析：

iptables 自带 `include/linux/netfilter/` 目录下的头文件（如 `xt_connmark.h`、`xt_mark.h`、`xt_dscp.h`）为空壳 wrapper。由于 include 搜索路径顺序问题，这些 wrapper 遮蔽了 sysroot 中包含真实结构体定义的版本。

以 `xt_connmark.h` 为例，include 链路如下：

- 源文件 `#include <linux/netfilter/xt_CONNMARK.h>` (大写)
- sysroot 的 `xt_CONNMARK.h` guard `_XT_CONNMARK_H_target` → 包含 `<linux/netfilter/xt_connmark.h>`

3. iptables wrapper 的 `xt_connmark.h` 被找到，但其 guard 同为 `_XT_CONNMARK_H_target`（已定义）→ 跳过
4. 结果：sysroot 的 `xt_connmark.h`（含结构体定义）永远无法被包含

解决方案（两步）：

1. 将 `$root_out_dir/sysroot/usr/include` 添加到 `include_dirs` 最前面，确保 sysroot 头文件优先被搜索
2. 删除 iptables 自带的 3 个冲突 wrapper 文件，彻底消除遮蔽

问题 4：D_SHIFT_LEFT 枚举重复声明

错误信息：

```
error: redeclaration of enumerator 'D_SHIFT_LEFT'
```

原因： `libxt_connmark.c` 源文件中定义了 `D_SHIFT_LEFT/D_SHIFT_RIGHT` 枚举，而 sysroot 的 `xt_connmark.h` 在删除 wrapper 后能被正确包含，也提供了相同的枚举定义。

解决方案： 从源文件中删除重复的枚举定义。

四、编译验证

编译命令

```
# 清理历史构建产物
rm -rf out

# 单独编译 iptables
hb build //third_party/iptables:iptables
```

编译结果

```
[OHOS INFO] build success
[OHOS INFO] Cost time: 0:03:19
```

生成文件

```
# 静态库
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/libs/
├─ libext.a          (714KB, ~60 个 libxt 扩展)
├─ libext4.a         (129KB, ~15 个 libipt 扩展)
├─ libext6.a         (191KB, ~20 个 libip6t 扩展)
├─ libxtables.a      (88KB, 核心库)
├─ libip4tc.a        (43KB)
└─ libip6tc.a        (43KB)

# 可执行文件
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/bin/
├─ iptables          (585KB)
└─ iptables-restore  (585KB)
```

文件信息

```
$ ls -lh out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/bin/iptables
-rwxrwxr-x 1 chenzhy223 chenzhy223 585K  5月 18 01:19 iptables

$ file out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/bin/iptables
iptables: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
```

文件特性:

- 文件大小: 585KB (stripped)
- 架构: ARM 64-bit (aarch64)
- 链接方式: 动态链接
- 解释器: musl libc (/lib/ld-musl-aarch64.so.1)
- PIE: 位置无关可执行文件

五、使用方法

基本功能测试

```
# 查看版本
iptables --version

# 列出所有规则
iptables -L -n

# 添加规则: 允许 SSH
iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# 删除规则
iptables -D INPUT -p tcp --dport 22 -j ACCEPT
```

扩展模块测试

注意: 以下扩展模块 (如 conntrack、mac、state 等) 需要内核编译对应的 `CONFIG_NETFILTER_XT_MATCH_XXX` 选项才能使用。原始内核 defconfig 仅启用了 5 个基本匹配 (mark、icmp、udplite、udp、tcp)。若需使用更多扩展, 参考 [iptables 内核扩展增强指南](#) 修改内核配置。

```
# TCP 端口匹配（内核内置，无需额外模块）
iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# UDP 端口匹配（内核内置，无需额外模块）
iptables -A INPUT -p udp --dport 53 -j ACCEPT

# ICMP 匹配（内核内置）
iptables -A INPUT -p icmp -j ACCEPT
```

以下命令需要启用对应的内核模块（参考 [内核扩展增强指南](#)）：

```
# MAC 地址匹配（需要 CONFIG_NETFILTER_XT_MATCH_MAC）
iptables -A INPUT -m mac --mac-source 00:11:22:33:44:55 -j DROP

# 连接跟踪（需要 CONFIG_NETFILTER_XT_MATCH_CONNTRACK）
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# 多端口匹配（需要 CONFIG_NETFILTER_XT_MATCH_MULTIPORT）
iptables -A INPUT -p tcp -m multiport --dports 22,80,443 -j ACCEPT
```

保存和恢复规则

```
# 保存规则
iptables-save > /tmp/rules.txt

# 清空规则
iptables -F

# 恢复规则
iptables-restore < /tmp/rules.txt
```

六、修改文件清单

修改的文件

文件	修改内容
vendor/hisilicon/hispark_hi3519dv500_linux/config.json	添加 iptables 组件到 thirdparty 子系统
third_party/iptables/extensions/BUILD.gn	移除 -D_UAPI_LINUX_SOCKET_H 和 -D__kernel_sa_family_t=sa_family_t 宏；所有 include_dirs 首位添加 sysroot 路径
third_party/iptables/extensions/libxt_connmark.c	删除重复的 D_SHIFT_LEFT/D_SHIFT_RIGHT 枚举定义

删除的文件

文件	删除原因
third_party/iptables/include/linux/netfilter/xt_connmark.h	空壳 wrapper，遮蔽 sysroot 的 struct xt_connmark_tginfo2 定义

文件	删除原因
<code>third_party/iptables/include/linux/netfilter/xt_mark.h</code>	空壳 wrapper，遮蔽 sysroot 的 struct xt_mark_tginfo2 定义
<code>third_party/iptables/include/linux/netfilter/xt_dscp.h</code>	自引用 wrapper，遮蔽 sysroot 的 XT_DSCP_MAX 定义

目录操作

操作	说明
确认 <code>third_party/iptables</code> 含完整源码	若为空壳，需获取完整移植版源码替换

七、与其他组件的关系

依赖关系

- **musl libc**：iptables 依赖 musl C 标准库（动态链接 `/lib/ld-musl-aarch64.so.1`）
- **Linux kernel headers**：需要内核头文件（netlink、netfilter、socket 等），通过 sysroot 提供
- **libmnl**：已整合到 hi3519dv500 平台，iptables 的部分扩展模块可能间接依赖

配合使用

iptables 可与已整合的其他网络组件配合使用：

1. 与 BusyBox 配合：

```
# 使用 busybox 的网络工具查看接口
busybox ifconfig -a
# 配置 iptables 规则
iptables -A INPUT -i eth0 -j ACCEPT
```

2. 与 ethtool 配合：

```
# 查看网卡状态
ethtool eth0
# 基于接口配置防火墙
iptables -A INPUT -i eth0 -j ACCEPT
```

八、注意事项

1. 架构差异

本产品（hispark_hi3519dv500_linux）使用 ARM64 架构，与 hispark_taurus_linux（ARM 32-bit）存在差异：

- 编译工具链：`aarch64-v01c01-linux-musl-gcc`
- 动态链接器：`/lib/ld-musl-aarch64.so.1`
- GCC 编译器（非 clang），部分 clang 专用编译选项（如 `-wno-tautological-pointer-compare`）会产生警

告但不影响编译

2. iptables 版本说明

- `third_party/iptables`: 完整移植版 (含 BUILD.gn、源码、genInit.py 等), 已自带 `ohos_lite` 分支

3. sysroot 头文件遮蔽问题

iptables 自带的 `include/linux/netfilter/` 部分头文件为空壳 wrapper, 会遮蔽 sysroot 中的完整版本。此问题仅在 ARM64 + musl 环境下暴露 (taurus 平台使用 clang + glibc 未遇到)。若后续更新 iptables 版本, 需重新检查并处理此类 wrapper 文件。

4. 脚本权限

通过 Windows 挂载的代码目录, 脚本文件需要手动添加执行权限。建议在构建前执行权限修复。

5. 内核模块依赖

iptables 扩展模块 (如 conntrack、mac、multiport、string 等) 需要内核编译对应的 `CONFIG_NETFILTER_XT_MATCH_XXX` 选项才能在运行时使用。用户空间编译正确不代表扩展可用, 还取决于内核配置。原始 hi3519dv500 内核 defconfig 仅启用了 5 个基本匹配 (mark、icmp、udplite、udp、tcp)。启用更多扩展的方法参见 [iptables 内核扩展增强指南](#)。

检查当前设备支持的内核匹配列表:

```
cat /proc/net/ip_tables_matches
```

6. 安全考虑

- iptables 涉及系统网络安全配置, 需谨慎配置规则
- 在生产环境中, 建议遵循最小权限原则配置防火墙规则
- 确保 iptables 二进制文件权限正确设置

九、参考资料

1. [iptables 官方文档](#)
2. [iptables v1.8.7 源码](#)
3. [OpenHarmony 编译构建指南](#)
4. [GN 构建系统参考](#)
5. [iptables 内核扩展增强指南](#) (启用 conntrack 等扩展所需的内核配置修改)